

Chapter 2: Representing Chemical Space (Small Molecules)

Chapter 1 introduced Lipinski’s Rule of Five and small-molecule representations only in passing, while surveying the two modalities this book covers. This chapter makes those representations precise. Much of what follows for small molecules — QSAR/ADMET models (Chapter 5), graph neural networks (Chapter 6), generative models over molecular strings and graphs (Chapter 7), and later, molecular docking (Chapter 11) — starts from one of the representations introduced here, so the central question this chapter answers is not just *how* to represent a molecule computationally, but how that choice of representation determines what “similar” can even mean.

2.1 Textual Representations: SMILES, SELFIES, and InChI

The simplest computational representation of a small molecule is a linear string. **SMILES** (Simplified Molecular Input Line Entry System; Weininger, 1988) encodes a molecular graph as a depth-first traversal: atoms are written as their element symbols (aromatic atoms in lowercase), bonds are implicit (single/aromatic) or explicit (= double, # triple), branches are parenthesized, and ring closures are marked with matching digits at the two atoms that close the ring. Aspirin, for example, is:

```
CC(=O)OC1=CC=CC=C1C(=O)O
```

reading as: a methyl carbon, a carbonyl carbon ((=O)), an ester oxygen, then a benzene ring opened and closed with the digit 1, with a carboxylic acid group closing the string. RDKit’s `Chem.MolFromSmiles` parses exactly this grammar into a molecule object — the same object the rest of this chapter’s code operates on.

Two properties of SMILES matter for everything that follows. First, it is **not unique by default**: a depth-first traversal can start at any atom and branch in any order, so the same molecule has many valid SMILES strings, all of which parse to the same graph. Toolkits like RDKit resolve this by producing a single *canonical* SMILES per molecule, using a deterministic atom-ranking algorithm, so that string equality can be used as a fast (if fingerprint-agnostic) exact-match test. Second, and more consequentially for this chapter, SMILES syntax does not track chemical *similarity* — a single-character edit near a ring-closure digit can produce a wildly different molecule or an invalid string entirely, so string-edit-distance between two SMILES is not a meaningful similarity metric. This motivates §2.2’s fingerprint-based approach directly: similarity has to be computed from *chemical structure*, not from surface-level string syntax.

SMILES’ non-uniqueness and syntactic fragility are also a practical liability for the generative models covered in Chapter 7: a character-level model sampling SMILES strings can easily produce syntactically invalid output (unbalanced rings, valence violations). **SELFIES** (Self-referencing Embedded Strings; Krenn et al., 2020) was designed specifically to close this gap — its grammar guarantees that *every* string it can produce decodes to a chemically valid molecule, at the cost of being less directly human-readable than SMILES. This book uses SMILES throughout Parts I–II for its readability and RDKit-native tooling, but Chapter 7 returns to SELFIES specifically where 100% validity matters for generation.

A third representation, the **InChI** (IUPAC International Chemical Identifier; Heller et al., 2015), solves a different problem: it is a non-proprietary, algorithmically-derived, canonical identifier layered with explicit sub-layers for formula, connectivity, tautomeric state, charge, and stereochemistry. Unlike a toolkit-specific canonical SMILES, an InChI (and its hashed, fixed-length InChIKey) is reproducible across independent implementations, which is exactly why databases such as ChEMBL and PubChem (Chapter 4) use it to deduplicate and cross-reference compounds submitted from different sources.

None of these three representations is strictly “better” — they solve different problems. SMILES is compact and directly parseable into a molecular graph; SELFIES trades some readability for guaranteed validity in generative settings; InChI trades human-writability for cross-database, implementation-independent identity. What none of them gives you directly is a numeric handle for comparing two *different* molecules — that is the job of the fingerprints introduced next.

2.2 Molecular Fingerprints and Similarity Metrics

From structure to bit vector

A **molecular fingerprint** encodes structural information as a fixed-length vector, so that two molecules can be compared with ordinary vector arithmetic instead of graph isomorphism. Fingerprints fall into two broad families.

Structural-key fingerprints fix a dictionary of specific substructures in advance and set one bit per dictionary entry if that substructure is present. The MACCS keys (Durant et al., 2002) are the canonical example: a 166-bit vector, each bit corresponding to a named, human-inspectable substructural pattern (e.g., “has an aromatic ring,” “has a carbonyl”). Their strength is direct interpretability — a set bit has a fixed, inspectable meaning; their limitation is that vocabulary is fixed, so structural motifs outside the predefined dictionary are invisible to the fingerprint no matter how chemically relevant they are.

Circular (Morgan/ECFP) fingerprints take the opposite approach: instead of a fixed dictionary, they enumerate the actual substructural environments present in the molecule, algorithmically. The underlying algorithm is Morgan’s (1965) method for canonical atom relabeling, originally designed to test structures for graph isomorphism at Chemical Abstracts Service: each atom is assigned an initial integer invariant (derived from its atomic number, degree, charge, attached hydrogen count, and ring membership), and then, over successive iterations, each atom’s invariant is recomputed as a hash of its own current invariant together with its neighbors’ invariants. Rogers and Hahn (2010) adapted this iterative relabeling procedure into a similarity fingerprint — **Extended-Connectivity Fingerprints (ECFP)** — by recording every atom-environment identifier produced at every iteration, rather than discarding intermediate values in pursuit of a single canonical numbering.

The number of iterations sets the fingerprint’s **radius**: after r iterations, an atom’s identifier summarizes its bonded environment out to r bonds away. ECFP4 — the fingerprint this chapter’s hands-on project uses — takes its name from a *diameter* of 4 (radius $r = 2$), so it captures each atom’s 2-bond neighborhood; ECFP6 ($r = 3$) captures larger, more global neighborhoods at the

Cosine coefficients, which weight shared bits differently — has become the de facto standard for binary fingerprint comparison in cheminformatics: it is simple, satisfies the mathematical properties of a proper distance metric ($1 - T$ is a true metric), and its behavior with respect to fingerprint density and molecule size is well characterized empirically.

In practice, similarity search requires choosing a **threshold** above which two molecules are treated as “similar enough” to act on — Martin et al.’s ≥ 0.85 figure above is one commonly cited reference point for ECFP-family fingerprints, but the appropriate threshold is target-, library-, and fingerprint-dependent, not a universal constant; Willett, Barnard, and Downs (1998), in their foundational review of similarity searching, make the same point about threshold sensitivity more generally.

Computing Tanimoto similarity for one query against a library of N compounds is $O(N)$ bitwise-AND/popcount operations — trivial for this chapter’s 17-molecule demo library, and still fast (fixed-width bitwise operations are cheap) against ChEMBL’s roughly 2.9 million compounds (Chapter 1, §1.4) for a single query. Where similarity search becomes computationally interesting is *many-query* or *all-pairs* search against million-compound libraries, which is where production systems move from brute-force scanning to indexing structures (e.g., inverted indices over set fingerprint bits, or approximate nearest-neighbor structures) that prune the candidate set before computing exact Tanimoto scores. This chapter’s hands-on project uses brute-force search deliberately — at demo scale, the indexing overhead is not worth paying, and the brute-force version keeps the code’s connection to §2.2’s mathematics direct and auditable.

2.3 Graph Representations

Underneath both the SMILES parser in §2.1 and the fingerprint generator in §2.2, RDKit represents every molecule the same way: as a graph $G = (V, E)$, with atoms as nodes V and bonds as edges E . Each node carries features — atomic number, formal charge, hybridization, aromaticity, attached hydrogen count — and each edge carries a bond order and ring membership flag. This graph is exactly what Morgan’s algorithm (§2.2) traverses to build atom-environment identifiers, and exactly what a conformer embedding algorithm (§2.4) uses as the connectivity constraints for 3D placement.

Formally, a molecular graph with n atoms can be described by an $n \times n$ **adjacency matrix** $\mathbf{A}$, where $\mathbf{A}_{ij}$ is nonzero if atoms i and j are bonded (and can encode bond order rather than a simple 0/1), together with a node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ holding each atom’s d -dimensional feature vector. This $(\mathbf{A}, \mathbf{X})$ pair is a strictly more complete representation than a fingerprint: a hashed fingerprint compresses the graph into a fixed-size, lossy summary optimized for cheap comparison, while $(\mathbf{A}, \mathbf{X})$ preserves the full topology, at the cost of a variable-size representation that cannot be compared with simple vector arithmetic. That trade-off — a fast, lossy fixed-size summary versus a complete, variable-size structure — is precisely what motivates learning similarity directly from $(\mathbf{A}, \mathbf{X})$ with graph neural networks, which Chapter 6 covers in full; this section introduces only the representation itself; the message-passing machinery that operates on it belongs there.

2.4 3D Conformations

Every representation covered so far — SMILES, InChI, fingerprints, molecular graphs — describes molecular **constitution**: which atoms are present and how they are bonded. None of them fixes a 3D shape. A single SMILES string can correspond to many distinct low-energy 3D arrangements of the same atoms (**conformers**), related by rotation around single bonds, and for a sufficiently flexible molecule the number of accessible conformers grows combinatorially with the number of rotatable bonds.

RDKit generates 3D conformers with **ETKDG** (Experimental-Torsion basic Knowledge Distance Geometry; Riniker & Landrum, 2015), which improves on classical distance-geometry embedding by biasing the initially-random atomic coordinates with experimentally observed torsion-angle preferences before refining them, substantially reducing the fraction of generated conformers that require heavy post-hoc correction. The embedded structure is typically then relaxed with a molecular-mechanics force field (RDKit supports both MMFF94 and UFF) to a local energy minimum. This chapter’s hands-on project works entirely in 2D (fingerprints and Lipinski descriptors do not require 3D coordinates), so it does not call RDKit’s conformer-generation API — but 3D conformers are the direct input to Chapter 11’s docking methods and a routine intermediate step for 3D-descriptor calculation, so it is worth knowing this is where they come from and why one SMILES is not one shape.

2.5 Hands-on Project: Molecular Similarity Search & Lipinski Filtering

The project code lives in this chapter’s folder (ch02_molecular_similarity/) and implements exactly the two ideas built up above: rank a small compound library by ECFP4/Tanimoto similarity to a query molecule (§2.2), then flag which of those candidates satisfy Lipinski’s Rule of Five.

Fingerprint generation and ranking

`similarity_search.py` builds one shared Morgan fingerprint generator,

```
FP_GENERATOR = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=2048)
```

— radius 2 (i.e., ECFP4, per §2.2’s radius/diameter convention) and a 2048-bit vector — and reuses it for both the query molecule and every molecule in `molecules.csv` (17 well-known drugs and biomolecules: aspirin, ibuprofen, caffeine, nicotine, dopamine, warfarin, and others). `rank_by_similarity` parses each SMILES with `Chem.MolFromSmiles`, generates its fingerprint, scores it against the query with `TanimotoSimilarity` exactly as defined in §2.2, and sorts the library in descending order of similarity. Unparsable library entries are skipped with a warning rather than crashing the run — a small but deliberate reproducibility choice, since a single malformed row in a larger, real-world compound library (say, a ChEMBL export) should not take down an entire screening job.

Lipinski filtering

`lipinski_pass` computes the four descriptors behind Lipinski’s Rule of Five (Lipinski et al., 2001), introduced in Chapter 1, §1.3, and applies it here in full:

$$\text{violations} = \mathbb{1}[\text{MW} > 500] + \mathbb{1}[\log P > 5] + \mathbb{1}[\text{HBD} > 5] + \mathbb{1}[\text{HBA} > 10]$$

with a molecule flagged as passing when violations ≤ 1 — the rule’s own stated tolerance for one violation, not a stricter pass/fail-on-any-violation reading. Each threshold is a multiple of five (hence the rule’s name): molecular weight under 500 Da, a calculated octanol-water partition coefficient ($\log P$) under 5, no more than 5 hydrogen-bond donors, and no more than 10 hydrogen-bond acceptors — an empirical description, from an analysis of compounds that reached Phase II clinical trials, of the property ranges associated with acceptable oral absorption and permeability.

The four descriptors are computed with RDKit as follows, and it is worth being precise about which specific method each one calls, since “log P” or “H-bond donor” is not a single universally agreed definition:

- **Molecular weight** — `Descriptors.MolWt`, the standard average molecular weight computed directly from the parsed structure.
- **LogP** — `Crippen.MolLogP`, an atom-contribution method (Wildman & Crippen, 1999) that estimates $\log P$ by summing per-atom contributions parameterized by each atom’s element and local environment, rather than measuring it experimentally.
- **H-bond donors and acceptors** — `Lipinski.NumHDonors` and `Lipinski.NumHAcceptors`, which implement RDKit’s own pattern-based definitions matched to Lipinski’s original paper, not a strict IUPAC donor/acceptor count. This distinction matters for reproducibility: a different toolkit’s H-bond counting rules can legitimately disagree with RDKit’s on edge cases (e.g., certain heteroaromatic acceptors), even though both claim to implement “Lipinski’s Rule of Five.”

Running it and reading the output

```
cd ch02_molecular_similarity
pip install -r requirements.txt
python similarity_search.py --query "CC(=O)OC1=CC=CC=C1C(=O)O" --top 5
```

The default query is aspirin; `print_table` reports each candidate’s name, Tanimoto similarity, molecular weight, LogP, H-bond donor/acceptor counts, and a PASS/FAIL Lipinski flag, sorted by similarity. Running the default query, the top result is aspirin itself, at Tanimoto similarity 1.000 (a molecule is, trivially, maximally similar to itself under §2.2’s definition) — a basic but genuine self-consistency check that the pipeline is doing what §2.2 says it should. Salicylic acid, aspirin’s close structural relative (aspirin is salicylic acid’s acetate ester), reliably ranks near the top as well, which is the qualitative behavior the similarity-property principle predicts for a close structural analogue.

Reproducibility

Dependencies are version-floored rather than exact-pinned (`rdkit>=2023.9.1` in `requirements.txt`, validated against 2026.3.5), since this project only touches stable, long-standing RDKit APIs (`rdFingerprintGenerator`, `Crippen`, `Descriptors`, `Lipinski`). The 12-test suite in `tests/test_similarity_search.py` checks `load_library`, `rank_by_similarity`, and `lipinski_pass` against real RDKit output rather than hardcoded expectations — including an end-to-end check that the self-match case returns Tanimoto 1.000, and a deliberately constructed long-PEG-chain SMILES that genuinely fails Lipinski on two independent axes (molecular weight

and H-bond acceptor count), so the “fail” branch is exercised by an actual RDKit computation rather than an invented one. `pip install -r requirements-dev.txt && pytest` from this chapter’s folder reproduces all 12 results.

From a 17-compound demo to a real screen

At 17 compounds, this project is a toy — its value is in making the mechanics of §2.2’s Tanimoto computation and §2.5’s Lipinski filter fully auditable end to end. The same two functions, unmodified, are the core operation behind real similarity-based **virtual screening**: given a known active (a “hit”), rank a real compound library — ChEMBL’s ~2.9 million compounds (Chapter 1, §1.4), or a proprietary corporate collection — by fingerprint similarity to surface candidates worth testing next. The same operation, run against molecules built on different ring systems or scaffolds than the query but landing at similarly high Tanimoto scores through shared substituent patterns, is the starting point for **scaffold hopping** — deliberately searching for structurally novel analogues of a hit, often to design around a patent or improve a liability like poor solubility while preserving the activity-relevant substructure. Both uses inherit the same caveat from §2.2: a high-ranked candidate is worth testing, not something already known to work — recall Martin et al.’s ~30% figure. Lipinski filtering plays a complementary, orthogonal role in this pipeline: it says nothing about whether a candidate will bind the target, only whether it is *shaped like* the historical population of orally bioavailable drugs — a cheap early filter to deprioritize candidates before spending assay budget on them, not a substitute for testing binding or activity directly.

Limitations of 2D similarity, and what comes next

Both of this chapter’s core tools — Tanimoto similarity over 2D fingerprints, and Lipinski’s Rule of Five — operate entirely on molecular **constitution**: which atoms are bonded to which, with no reference to three-dimensional shape, a specific target’s binding-pocket geometry, or measured biological activity. That is a deliberate scope, not an oversight, but it has three concrete consequences for how these tools should — and should not — be used going forward in this book:

1. **2D similarity does not see 3D shape.** Two molecules can score a high Tanimoto similarity while adopting substantially different 3D conformations (§2.4), and, in the other direction, two molecules built on entirely different 2D scaffolds — and therefore scoring a *low* Tanimoto similarity — can present near-identical 3D pharmacophores to a binding pocket. The first is a false positive for “these will behave alike”; the second is a false negative that a pure 2D similarity search will simply never surface. Reasoning about actual binding requires the protein-side structural representations Chapter 3 introduces, and culminates in the explicit 3D docking methods of Chapter 11.
2. **Lipinski’s Rule of Five predicts drug-likeness, not activity or safety.** Passing it says a molecule’s bulk physicochemical properties resemble those of historically orally-bioavailable drugs; it says nothing about whether the molecule binds any particular target, is potent, or is toxic. Predicting those properties from structure is the job of the supervised QSAR and ADMET models in Chapter 5, trained on labeled bioactivity and toxicity data rather than on a four-descriptor heuristic.
3. **Both tools are static.** A fingerprint and a Lipinski pass/fail are computed once, from one fixed structure (or, for Lipinski, one conformer-independent set of descriptors); neither says

anything about how a molecule and a target move and interact over time. That dynamic picture — binding kinetics, conformational flexibility, stability of a docked pose — is the subject of the molecular dynamics methods in Chapter 12.

None of this makes 2D similarity search or Lipinski filtering wrong tools — used as what they are, a cheap first-pass filter over enormous chemical spaces, they remain standard practice throughout the industry precisely because of their speed. The point is narrower: treat a high similarity score or a Lipinski pass as a reason to *look closer* with the structure-, activity-, and dynamics-aware methods later in this book, not as a conclusion in itself.

A note on Google Colab

rdkit installs on Google Colab’s default runtime with a single `pip install rdkit`; the rest of this project’s dependencies are standard library. No GPU is required — every part of this chapter’s project runs on Colab’s free-tier CPU runtime unmodified.

References

- Weininger, D. (1988). SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1), 31–36. <https://doi.org/10.1021/ci00057a005>
- Krenn, M., Häse, F., Nigam, A., Friederich, P., & Aspuru-Guzik, A. (2020). Self-referencing embedded strings (SELFIES): A 100% robust molecular string representation. *Machine Learning: Science and Technology*, 1(4), 045024. <https://doi.org/10.1088/2632-2153/aba947>
- Heller, S. R., McNaught, A., Pletnev, I., Stein, S., & Tchekhovskoi, D. (2015). InChI, the IUPAC International Chemical Identifier. *Journal of Cheminformatics*, 7, 23. <https://doi.org/10.1186/s13321-015-0068-4>
- Morgan, H. L. (1965). The generation of a unique machine description for chemical structures - a technique developed at Chemical Abstracts Service. *Journal of Chemical Documentation*, 5(2), 107–113. <https://doi.org/10.1021/c160017a018>
- Rogers, D., & Hahn, M. (2010). Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5), 742–754. <https://doi.org/10.1021/ci100050t>
- Durant, J. L., Leland, B. A., Henry, D. R., & Nourse, J. G. (2002). Reoptimization of MDL keys for use in drug discovery. *Journal of Chemical Information and Computer Sciences*, 42(6), 1273–1280. <https://doi.org/10.1021/ci010132r>
- Bajusz, D., Rácz, A., & Héberger, K. (2015). Why is Tanimoto index an appropriate choice for fingerprint-based similarity calculations? *Journal of Cheminformatics*, 7, 20. <https://doi.org/10.1186/s13321-015-0069-3>
- Willett, P., Barnard, J. M., & Downs, G. M. (1998). Chemical similarity searching. *Journal of Chemical Information and Computer Sciences*, 38(6), 983–996. <https://doi.org/10.1021/ci9800211>
- Martin, Y. C., Kofron, J. L., & Traphagen, L. M. (2002). Do structurally similar molecules have similar biological activity? *Journal of Medicinal Chemistry*, 45(19), 4350–4358. <https://doi.org/10.1021/jm020155c>
- Wildman, S. A., & Crippen, G. M. (1999). Prediction of physicochemical parameters by atomic contributions. *Journal of Chemical Information and Computer Sciences*, 39(5), 868–873. <https://doi.org/10.1021/ci990307l>

- Riniker, S., & Landrum, G. A. (2015). Better informed distance geometry: using what we know to improve conformation generation. *Journal of Chemical Information and Modeling*, 55(12), 2562–2574. <https://doi.org/10.1021/acs.jcim.5b00654>

See Chapter 1’s references for Lipinski, C. A., Lombardo, F., Dominy, B. W., & Feeney, P. J. (2001), Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings — reused here rather than re-listed. RDKit itself has no official journal publication; its maintainers’ recommended citation is “RDKit: Open-source cheminformatics. <https://www.rdkit.org>” (confirmed directly from the project’s own documentation before use here), matching the convention already established in Chapter 4’s references.

Johnson, M. A., & Maggiora, G. M. (Eds.). (1990). *Concepts and Applications of Molecular Similarity*. Wiley — cited above for the similarity-property principle terminology; a book, not a journal article, so listed separately from the verified journal citations above.